

Simulating the Classical Distributions

A working tour from uniform bits to the t -test,
with the `probsim` C99 library

The `probsim` project

July 2026

Abstract

This paper accompanies `probsim`, a small C99 library that simulates the twelve classical probability distributions and carries the results all the way to working t -tests. For each distribution we explain what it models, where it earns its keep in statistical inference and applied modeling, and how the library simulates it — always with the simplest algorithm that is still practical. The distributions are presented as one connected family (Figure 1): every sampler is built from uniform bits, and the family funnels naturally into Student’s t , whose tail areas turn simulated or observed data into p -values. Every section names the source file and function that implements it, and the sources cite these sections back, so paper and code can be read side by side. This paper also has [an interactive web edition](#) (`docs/index.html`, served via GitHub Pages) with every figure animated — each figure caption below links to its interactive counterpart.

1 Introduction

A probability distribution is a reusable answer to the question “what does this kind of randomness look like?” A handful of them — the *classical* distributions — cover a remarkable share of practical work: coin-like events, counts, waiting times, measurement noise, and the sampling behaviour of statistics computed from data. Learning them as isolated formulas is hard; learning them as a *family*, where each new member is a short construction from ones you already know, is much easier. That family view is exactly how the `probsim` library is written, and Figure 1 is its map.

How to read this with the code. The library is four small C files behind one header, `include/probsim.h`. Section §2 covers `src/rng.c`; §3 and §4 cover `src/dist.c`; §5 covers `src/special.c` and `src/stats.c`; §6 walks through the driver `app/simulate.c` and the test suite `tests/test_probsim.c`. Each subsection below ends with a pointer to its implementation, and the source comments point back here. Throughout, the guiding rule is the one this library was written under: *prefer the simplest algorithm that is easy to understand and still practical to use*. Where a cleverer method is the industry standard — the ziggurat for normals, say (Marsaglia and Tsang, 2000b) — we cite it, explain the trade-off, and keep the simple one.

2 Where the randomness comes from

Computers are deterministic, so “random” numbers are produced by a *pseudo-random number generator* (PRNG): a small state machine whose output sequence is indistinguishable, for statistical purposes, from true randomness. `probsim` uses `xoshiro256++` (Blackman and Vigna, 2021): 256 bits of state, a period of $2^{256} - 1$, excellent results on the standard test batteries, and a step function of about ten lines — comfortably inside our simplicity rule.

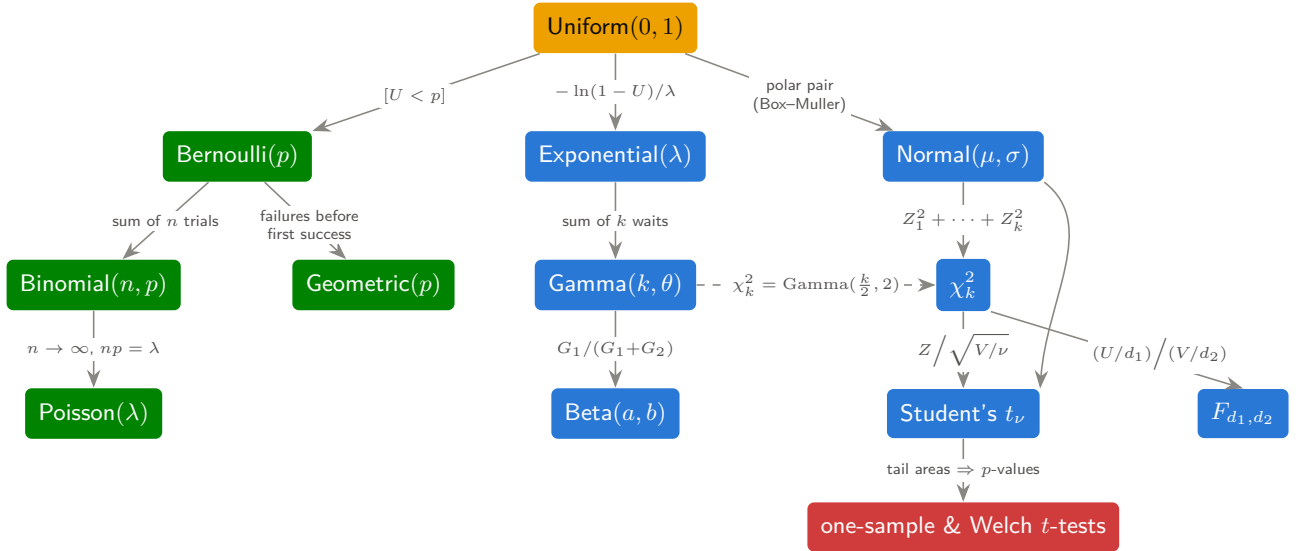


Figure 1: The classical family as **probsim** builds it. Everything starts from uniform bits (§2); **discrete** distributions grow along the left (§3), **continuous** ones along the right (§4), and the family funnels into the **t-tests** of §5. Each arrow is literally a line or two of C in `src/dist.c`. Three less-common relatives for hash modeling hang off this tree: the negative binomial (§3.5) is a sum of geometric waits, and the Rayleigh and Gumbel (§4.9–4.10) are one-line transforms of the exponential. [\[interactive version on the web\]](#)

Reasonable alternatives with the same spirit include PCG (O’Neill, 2014); the deeper theory of testing generators is in Knuth (1997, ch. 3). None of these are cryptographic generators, and **probsim** should never be used where an adversary matters.

Two practical details deserve attention because they bite working statisticians. First, *seeding*: naive seeds like 1, 2, 3 are nearly identical bit patterns, and a weak seeding routine would start the streams in correlated states. The library therefore runs every seed through the `splitmix64` mixer, as the `xoshiro` authors recommend. Second, *explicit state*: every sampler takes a `ps_rng *` argument and there is no hidden global, so a simulation is reproducible from a single recorded seed and independent streams are just independent structs — which is what makes the test suite of §6 deterministic.

2.1 Uniform is enough

The raw generator emits 64-bit integers; dividing the top 53 bits by 2^{53} gives a double U uniform on $[0, 1)$ at full mantissa precision. Everything else in this paper is manufactured from such U ’s. The master tool is *inverse-transform sampling*: if F is a CDF and $U \sim \text{Uniform}(0, 1)$, then $X = F^{-1}(U)$ has CDF exactly F , because $\Pr[X \leq x] = \Pr[U \leq F(x)] = F(x)$. Figure 2 shows the idea; the exponential sampler of §4.2 is its purest one-line application. When F^{-1} is awkward, two other levers — *transformation* (build the variable from simpler ones) and *rejection* (propose, then accept with the right probability) — take over; both appear in §4. The encyclopedia of all three techniques is Devroye (1986).

↪ implemented in `src/rng.c`: `ps_rng_seed()`, `ps_rng_next()`, `ps_runif()`

2.2 The generator under the microscope

Since every result in this library is only as trustworthy as its stream of uniforms, the generator deserves a closer look: what kind of object `xoshiro256++` is, which of its properties are *theorems*, which rest on *measurement*, and what remains open.

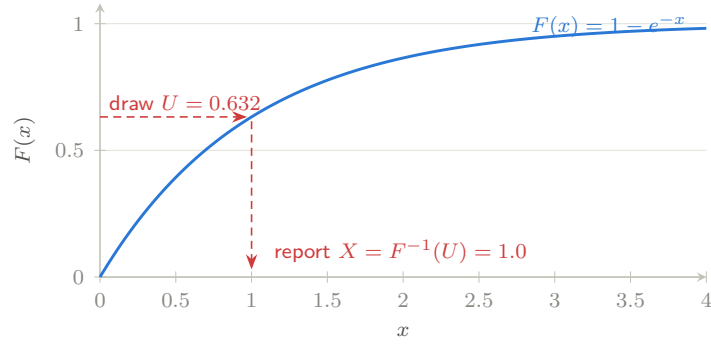


Figure 2: Inverse-transform sampling: a uniform draw on the vertical axis is pushed through F^{-1} to land on the horizontal axis with density exactly F' . Regions where F rises steeply receive more of the uniform mass — that is the whole trick. [[interactive version on the web](#)]

What it is. `xoshiro256++` belongs to the family of *scrambled linear* generators (Blackman and Vigna, 2021): a linear *engine* advances the 256-bit state $s = (s_0, s_1, s_2, s_3)$ using only XORs, shifts and rotations — a linear map over the two-element field $\mathbb{F}_2$ — and a small nonlinear *scrambler*, here $\text{rotl}(s_0 + s_3, 23) + s_0$ with 64-bit addition, hashes the state into each output word (Figure 3). The division of labour is deliberate. Linear maps are the one class of generators whose long-run structure mathematics can fully analyse, but their raw outputs carry visible linear fingerprints; the scrambler destroys those artifacts while provably not shortening the cycle, since it never feeds back into the state. The design refines a lineage that begins with Marsaglia’s xorshift generators (Marsaglia, 2003) and their systematic re-examination in Vigna (2016).

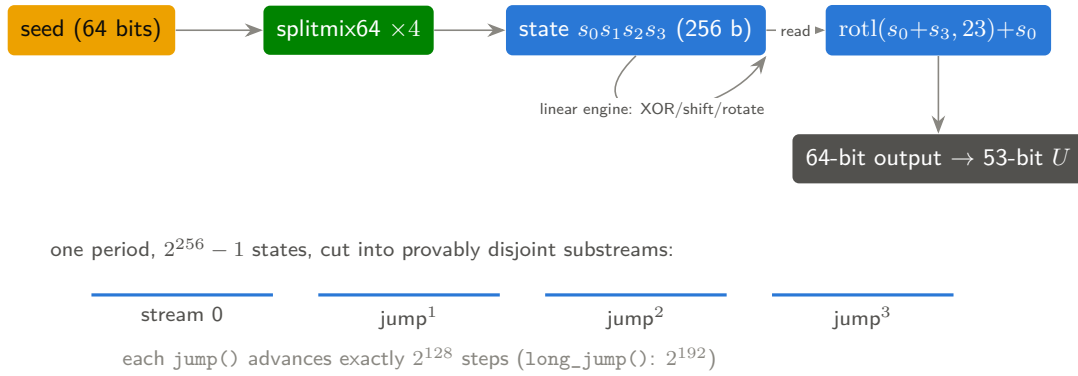


Figure 3: Anatomy of `xoshiro256++`: a seed is spread over 256 bits of state by `splitmix64`; an $\mathbb{F}_2$ -linear engine walks a single cycle of length $2^{256} - 1$; a nonlinear scrambler hashes each state into the output word whose top 53 bits become the uniform draw of §2.1. Jump functions cut the cycle into provably non-overlapping substreams for parallel simulation. [[interactive version on the web](#)]

What is proved. The engine’s transition matrix over $\mathbb{F}_2$ has a primitive characteristic polynomial, which yields the central theorem: the state sequence is one single cycle through *every* nonzero 256-bit state, so the period is exactly $2^{256} - 1$ (Blackman and Vigna, 2021). Two corollaries matter in practice. First, the all-zero state is the unique fixed point and must never be used — one reason `ps_rng_seed()` runs every seed through `splitmix64` rather than loading it directly. Second, because the engine is linear, jumping ahead by any fixed number of steps is a matrix power, computable exactly: the published `jump` and `long_jump` polynomials advance the

state by precisely 2^{128} and 2^{192} steps, giving parallel simulations substreams that are *guaranteed* never to overlap. On the output side, the authors use the theory of filtered linear-feedback shift registers to prove properties of the scramblers themselves, and for the sibling multiplicative scramblers ($*$ and $**$) they prove $n/64$ -dimensional equidistribution — four dimensions at this state size: every quadruple of consecutive outputs appears equally often over the full period.

What is measured. No practical non-cryptographic generator comes with a proof that it passes statistical tests, so the case for quality is empirical — and should be read the same way as any experiment. `xoshiro256++` shows *no systematic failures* on BigCrush, the most demanding battery in the TestU01 suite (L’Ecuyer and Simard, 2007), and no anomalies in long PractRand runs (Doty-Humphrey, 2014). More telling is a test the authors built precisely because the standard suites missed something: raw $\mathbb{F}_2$ -linear engines exhibit long-range dependencies in the *Hamming weight* (popcount) of their outputs, invisible to BigCrush. Their Hamming-weight dependency test detects such bias in unscrambled and weakly scrambled generators, while `xoshiro256++` passes it in the tested ranges (Blackman and Vigna, 2021) — the empirical confirmation that the scrambler is doing its job. The contrast with the family’s own history is instructive: the low-order bits of the simpler `xoshiro256+` inherit low linear complexity from the engine and fail linearity-sensitive tests, which is exactly why this library uses the `++` scrambler.

What is open, and what is simply false. Openness, first: there is no theorem that `xoshiro256++` passes any statistical test — for these generators such guarantees are structural (period, equidistribution, jumps) while test-passing is evidence, bounded by the compute anyone has spent; the $n/64$ equidistribution theorem is stated for the multiplicative scramblers, not for `++`, whose output equidistribution is not fully characterised; and a battery that fails it tomorrow is always possible in principle — that is the nature of empirical quality. Falsity, second: `xoshiro256++` is *not* a cryptographic generator. Its state-recovery resistance is nil by design — observing a modest number of outputs suffices to reconstruct the state and predict the stream — so it must never guard secrets, tokens or shuffles an adversary can bet against. For simulation, the working summary is comfortable: everything this paper’s simulations need — reproducibility, stream independence, full-period coverage, absence of detectable bias at our sample sizes — is either proved or measured with margin to spare.

↔ implemented in `src/rng.c` (engine, scrambler and seeding, with these properties cited)

3 The discrete family

Discrete distributions describe counts. The four classical ones form a single story: a yes/no event (Bernoulli), how many yeses in n tries (binomial), how long until the first yes (geometric), and how many events occur when opportunities are numerous but each is rare (Poisson). The driver `app/simulate.c` exercises each with the worked parameters used below.

3.1 Bernoulli

The $\text{Bernoulli}(p)$ variable is a single yes/no experiment: $\Pr[X=1] = p$, $\Pr[X=0] = 1 - p$, with $\mathbb{E}X = p$ and $\text{Var } X = p(1 - p)$. It is the atom of applied statistics: a conversion in an A/B test, a defective part, a patient responding to treatment, a loan defaulting. Almost all of *binary-outcome* inference — estimating a proportion, comparing two proportions, logistic regression — is Bernoulli modeling. Simulation is the definition itself: draw U and report whether $U < p$.

Origins and habitat. The distribution carries the name of Jacob Bernoulli, whose posthumous *Ars Conjectandi* (Bernoulli, 1713) proved the first law of large numbers — his “golden theorem” that the observed frequency of successes converges to p — and thereby turned the coin toss from a gambling prop into the founding object of mathematical statistics. The model arises naturally wherever an outcome is *recorded* as a dichotomy, which is a choice as much as a fact of nature: cure/no cure, click/no click, default/no default. Its independence-plus-constant- p idealisation is what everything downstream inherits, so the practical craft lies in checking it — runs of correlated outcomes or drifting p break the binomial counts and waiting times built on top. In modern practice the Bernoulli likelihood is the atom of logistic regression, of web-scale A/B platforms running thousands of simultaneous coin-flip experiments, and of the Beta–Bernoulli machinery of §4.5.

↪ implemented in `src/dist.c`: `ps_bernoulli()`

3.2 Binomial

The $\text{Binomial}(n, p)$ variable counts successes in n independent $\text{Bernoulli}(p)$ trials:

$$\Pr[X = k] = \binom{n}{k} p^k (1 - p)^{n-k}, \quad \mathbb{E}X = np, \quad \text{Var } X = np(1 - p).$$

It models any fixed-quota count: heads in n tosses, positive responses among n patients, defects in a batch of n (the basis of industrial *acceptance sampling*), conversions among n visitors. In inference it is the exact sampling distribution behind every test of a proportion — the sign test, the exact binomial test, and, through the normal approximation, the familiar z -test for proportions. `probsim` simulates it as what it is — n Bernoulli draws summed — an $O(n)$ loop that cannot be wrong by inspection; sub-linear samplers exist (Devroye, 1986, ch. X.4) but buy speed with complexity we do not need. Figure 4 (left) shows the worked example $\text{Binomial}(20, 0.25)$.

Origins and habitat. The binomial is probability theory’s birth certificate. The 1654 Pascal–Fermat correspondence on the *problem of points* — how to split the stakes of an interrupted game — amounted to computing binomial probabilities from Pascal’s triangle, and Bernoulli (1713) supplied the general trials framework. Most consequentially, de Moivre (1738) showed that for large n the binomial is approximated by the curve e^{-x^2} : the first central limit theorem, and the historical doorway through which the normal distribution entered statistics (§4.3). The model arises wherever a fixed number of like units each independently succeed or fail: Mendel’s famous 3:1 pea ratios are binomial counts; industrial *acceptance sampling* (developed by Dodge and Romig at Bell Laboratories in the 1920s–40s) decides a lot’s fate from the binomial behaviour of a small inspected sample; election audits and A/B tests are the same calculation with different stakes. The standard failure mode is *overdispersion* — p varying between units inflates the variance beyond $np(1 - p)$ — which is the empirical signature that the Beta–binomial mixture of §4.5 fits better.

↪ implemented in `src/dist.c`: `ps_binomial()`, `ps_binomial_pmf()`

3.3 Geometric

The $\text{Geometric}(p)$ variable counts *failures before the first success* (support $0, 1, 2, \dots$, the convention of R’s `rgeom`): $\Pr[X = k] = p(1 - p)^k$, $\mathbb{E}X = (1 - p)/p$, $\text{Var } X = (1 - p)/p^2$. It is the discrete waiting time: retries before a packet gets through, cold calls before a sale, coin flips before the first head. Its *memoryless* property (past failures tell you nothing about the future) makes it the discrete twin of the exponential — and indeed the sampler exploits exactly

that: invert the geometric CDF with one uniform, $X = \lfloor \ln(1 - U) / \ln(1 - p) \rfloor$, which is the exponential inversion of §4.2 rounded down to a count.

Origins and habitat. “How many throws until the first six?” is as old as dice questions get — de Méré’s puzzles that provoked the Pascal–Fermat correspondence were of exactly this waiting-time kind. Two properties give the geometric its character. It is provably the *only* discrete distribution with the memoryless property, so it arises, exactly, whenever a repeated trial has a constant per-attempt hazard; and the everyday refusal to believe this — “red is due” after a run of black — is the gambler’s fallacy, which makes the geometric the distribution most humans mis-intuit. It appears naturally as retransmission counts in randomized network protocols (the backoff analyses behind ALOHA and Ethernet), as cycles-to-conception in demography’s fecundability models, as the first defect’s position in sequential inspection, and as discrete-time survival with constant hazard — the null model against which real ageing or learning effects are measured.

↪ implemented in `src/dist.c`: `ps_geometric()`, `ps_geometric_pmf()`

3.4 Poisson

The $\text{Poisson}(\lambda)$ variable is the law of rare events: the limit of $\text{Binomial}(n, \lambda/n)$ as $n \rightarrow \infty$,

$$\Pr[X = k] = e^{-\lambda} \frac{\lambda^k}{k!}, \quad \mathbb{E}X = \text{Var } X = \lambda.$$

It models counts per unit of exposure when events are individually unlikely but opportunities are many: arrivals at a queue per minute, radioactive decays per second, typos per page, insurance claims per policy-year, sequencing reads per gene. In inference it anchors count regression (Poisson GLMs, with the $\mathbb{E}X = \text{Var } X$ property serving as the canonical check for *overdispersion*) and classical goodness-of-fit tests. The library simulates it with Knuth’s product-of-uniforms method (Knuth, 1997, §3.4.1) — count uniforms until their running product drops below $e^{-\lambda}$ — applied in chunks of rate at most 30 so the product never underflows; the chunking is valid because Poisson rates add. Figure 4 (right) shows $\text{Poisson}(4)$.

Origins and habitat. Poisson derived his limit in an 1837 treatise on the probability of criminal-jury verdicts, where it went largely unnoticed; it was von Bortkiewicz (1898), with his “law of small numbers”, who made it famous — his tabulation of deaths by horse kick across Prussian cavalry corps remains the canonical demonstration that rare-event counts follow the Poisson law with uncanny fidelity. Two independent rediscoveries cemented its physical and industrial standing: Rutherford and Geiger’s 1910 alpha-particle counts matched Poisson predictions and helped establish the randomness of radioactive decay, while Erlang’s 1909 analysis of telephone-call arrivals at the Copenhagen exchange founded teletraffic theory and, with it, queueing. The genesis condition is superposition: many independent sources, each unlikely to fire in any short interval, add up to Poisson counts — which is why the model recurs for insurance claim frequencies, mutations along a genome, web-server request arrivals, and photons at a detector. As with the binomial, real data often show variance exceeding the mean; that overdispersion, and the gamma-mixed Poisson (negative binomial) that absorbs it, is the standard second chapter of count modeling.

↪ implemented in `src/dist.c`: `ps_poisson()`, `ps_poisson_pmf()`

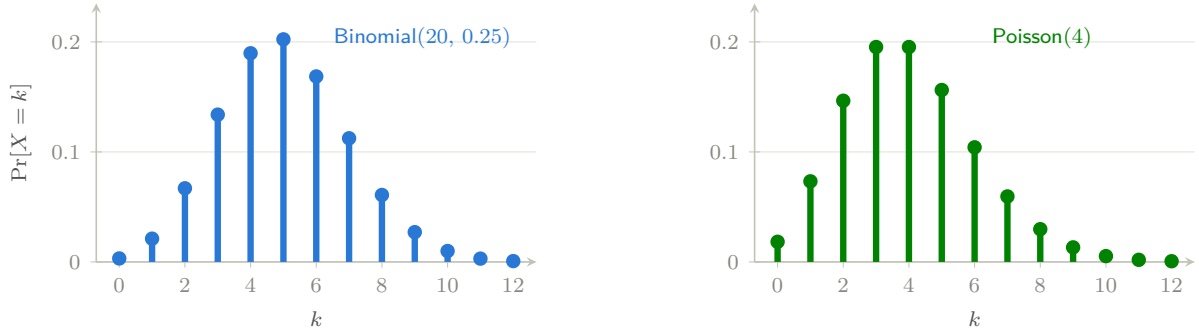


Figure 4: The two classical count laws, at the driver’s worked parameters. Both have mean $\approx 4\text{--}5$, but the binomial is confined to $\{0, \dots, 20\}$ while the Poisson has an unbounded right tail — the visible signature of “many opportunities, each rare”. [\[interactive version on the web\]](#)

3.5 Negative binomial

The negative binomial $\text{NB}(r, p)$ counts *failures before the r -th success*:

$$\Pr[X = k] = \binom{k+r-1}{k} p^r (1-p)^k, \quad \mathbb{E}X = \frac{r(1-p)}{p}, \quad \text{Var } X = \frac{r(1-p)}{p^2},$$

the geometric of §3.3 being the case $r = 1$ — and the sampler is that identity, a sum of r geometric waits. Its CDF is one more incomplete-beta reduction, $\Pr[X \leq k] = I_p(r, k+1)$ (§5.1). It wears two very different hats. As a *waiting time* it prices repeated search: if each independent attempt succeeds with probability p , the total work to collect r successes is NB-distributed — which, with p of order 2^{-n} , is exactly the cost model for finding r preimages or r collisions against an ideal n -bit hash, the multi-target attacks of §4.9’s annex. As a *count model* it is the gamma-mixed Poisson: let the Poisson rate of §3.4 itself vary as a gamma (§4.4) across units and the marginal count is negative binomial, with the variance exceeding the mean — the promised cure for overdispersion.

Origins and habitat. The pmf appears (as a gambling wait) in Montmort’s 1714 analysis of the problem of points, and the name records a formal joke: the coefficients come from the binomial series with a *negative* exponent. Its modern statistical career began with [Greenwood and Yule \(1920\)](#), who derived it as the law of accident counts among London munitions workers when individual proneness varies — the original overdispersion argument, gamma-mixing included — and it has been the default model for clumped counts ever since: insurance claims, species abundance in quadrats, crime counts, and today the per-gene read counts of RNA-seq, whose standard analysis tools are negative-binomial regressions at scale.

↪ implemented in `src/dist.c`: `ps_negative_binomial()`, `ps_negative_binomial_pmf()`, `ps_negative_binomial`

4 The continuous family

Continuous distributions describe measurements. We meet them in the order the library constructs them (the right-hand side of Figure 1), because each is a short recipe from the previous ones.

4.1 Uniform

$\text{Uniform}(a, b)$ spreads its mass evenly: density $1/(b-a)$ on $[a, b)$, mean $(a+b)/2$, variance $(b-a)^2/12$. Beyond being the raw material of §2.1, it is a model in its own right (rounding

error, phases, arrival offsets) and a cornerstone of inference through one beautiful fact: *under a true null hypothesis, a p -value is $\text{Uniform}(0, 1)$* . That fact is testable — the test suite verifies it empirically in §6.3 — and it underlies p -value histograms, false-discovery-rate methods, and simulation-based calibration. Simulation is one affine map of U .

Origins and habitat. The uniform is classical probability’s silent axiom: Laplace’s definition of probability as favourable cases over “equally possible” cases (Laplace, 1812) simply *is* the discrete uniform, elevated by his principle of insufficient reason into the default model of ignorance — a move philosophers have argued about ever since (equal in which parameterisation?). Its modern career began at Los Alamos in the late 1940s, when Ulam and von Neumann turned streams of uniform pseudo-random digits into the Monte Carlo method, making the uniform a computational instrument rather than a modeling assumption — the role it plays in every sampler of this library. In data it arises as quantization and rounding error, as the phase at which an unsynchronized observer catches a cycle, and universally through the *probability integral transform*: $F(X) \sim \text{Uniform}(0, 1)$ for any continuous F , the identity behind p -value uniformity, inverse-transform sampling (§2.1), and copula modeling, which separates each variable’s margin from the dependence between them.

↪ implemented in `src/dist.c`: `ps_uniform()`

4.2 Exponential

The $\text{Exponential}(\lambda)$ variable (rate λ ; density $\lambda e^{-\lambda x}$, mean $1/\lambda$, variance $1/\lambda^2$) is the continuous waiting time to the next event of a constant-rate process — the continuous twin of the geometric, and like it *memoryless*: a component that has survived t hours is statistically as good as new. That property makes it the baseline model of reliability engineering, queueing theory (service and inter-arrival times), and survival analysis, where its constant *hazard rate* is the reference against which ageing (or infant-mortality) effects are measured. The sampler is the one-liner promised in Figure 2: $X = -\ln(1 - U)/\lambda$.

Origins and habitat. The exponential earned physical reality around 1900, when Rutherford and Soddy found that radioactive decay follows an exact exponential law with a substance-specific half-life — memorylessness measured in a laboratory. Mathematically it is welded to the Poisson process: exponential inter-arrival times and Poisson counts are two descriptions of the same object, so Erlang’s telephone traffic and every queueing model since carry it as the default clock. Like its discrete twin, it is provably the *only* continuous memoryless distribution, and it is closed under competing risks — the minimum of independent exponentials is exponential with the rates summed — which is why systems of many independent failure modes look exponential in their prime of life. Post-war reliability engineering built its handbooks on exactly that constant-hazard middle of the “bathtub curve”, and modern survival analysis still starts from the exponential as the null against which ageing (Weibull), frailty, and cure models are judged.

↪ implemented in `src/dist.c`: `ps_exponential()`, `ps_exponential_cdf()`

4.3 Normal

The $\text{Normal}(\mu, \sigma)$ density $\frac{1}{\sigma\sqrt{2\pi}}e^{-(x-\mu)^2/2\sigma^2}$ is the center of gravity of statistics, for one overwhelming reason: the *central limit theorem*. Sums and averages of many small independent effects are approximately normal almost regardless of the ingredients’ own distributions — which is why measurement errors, biological traits, and sample means all look Gaussian, and why the normal is the default model for noise in regression and the reference distribution for estimators. Essentially all of §5 exists to make small-sample corrections to normal-based reasoning.

Origins and habitat. No distribution has a richer pedigree. [de Moivre \(1738\)](#) found the curve as the large- n limit of the binomial; [Gauss \(1809\)](#) derived it as the error law under which the arithmetic mean is the best estimate, founding least squares in the process (hence *Gaussian*); Laplace’s 1810 central limit theorem explained *why* it appears — sums of many small independent contributions converge to it almost regardless of their individual laws ([Laplace, 1812](#)). The nineteenth century then watched it colonise the empirical world: Quetelet fit it to human chest measurements and birthed social statistics’ “average man” (1835), Maxwell derived it for molecular velocities (1860), and Galton’s quincunx made the mechanism visible in falling beads; the now-standard name “normal” spread through Galton and Pearson late in the century — a naming historians note is unfortunately value-laden. Its natural habitats are exactly the CLT’s: measurement errors built from many small disturbances, biological traits summing many genetic and environmental effects, diffusive displacements (Einstein’s 1905 Brownian-motion analysis), and averages of almost anything. The perennial caution is in the tails: the CLT speaks about the middle of the distribution, and real-world extremes — finance above all — are routinely heavier than Gaussian, which is one job Student’s t (§4.7) takes over.

F^{-1} has no closed form, so inversion is unavailable; instead the library uses the polar variant of the Box–Muller transform ([Box and Muller, 1958](#)), our one *rejection* sampler (Figure 5): draw (v_1, v_2) uniformly in the square $[-1, 1]^2$, accept when $s = v_1^2 + v_2^2 < 1$ (probability $\pi/4 \approx 79\%$), and return $v_1\sqrt{-2\ln s/s}$, which is exactly standard normal.¹

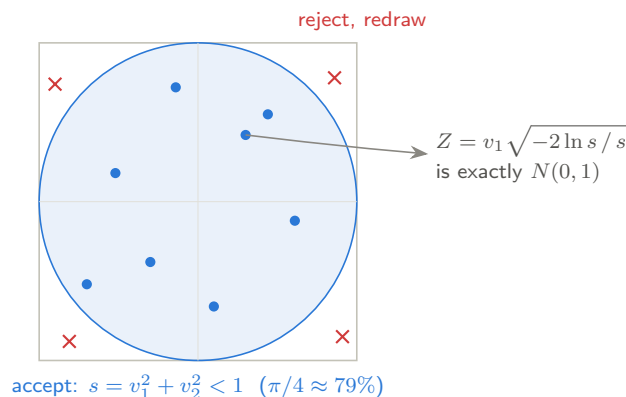


Figure 5: The polar Box–Muller sampler: uniform points in the square are kept only if they land in the disc; the kept point’s radius and direction are then reshaped into a standard normal. Rejection turns an easy distribution (uniform on a square) into a hard one (uniform on a disc) at a known, modest cost. [[interactive version on the web](#)]

↪ implemented in `src/dist.c`: `ps_normal()`, `ps_normal_pdf()`, `ps_normal_cdf()`

4.4 Gamma

The $\text{Gamma}(k, \theta)$ family (shape k , scale θ ; density $\propto x^{k-1}e^{-x/\theta}$, mean $k\theta$, variance $k\theta^2$) is the flexible law of *accumulated* waiting: for integer k it is exactly the time until the k -th event of a Poisson process — a sum of k exponentials — and general k interpolates smoothly (Figure 6). It models rainfall totals, insurance losses, service times, and neuronal inter-spike intervals; in Bayesian inference it is the conjugate prior for Poisson and exponential rates, so posterior simulation of rate parameters is gamma sampling. It also generates the next two distributions outright.

¹The method actually yields *two* independent normals, $v_1\sqrt{-2\ln s/s}$ and $v_2\sqrt{-2\ln s/s}$; `ps_normal()` discards the second so the sampler carries no state between calls. Production libraries chasing speed instead use the zigurat method ([Marsaglia and Tsang, 2000b](#)), a heavily engineered rejection scheme — a good example of the complexity our simplicity rule declines.

Summing exponentials only works for integer k , so the library uses the one algorithm where we accept a little cleverness for a lot of practical value: the [Marsaglia and Tsang \(2000a\)](#) method, the standard choice of modern statistical software, and still only a dozen lines. Its idea is a polished version of Figure 5’s: push a normal draw Z through the smooth cube $X = d(1 + Z/\sqrt{9d})^3$ with $d = k - \frac{1}{3}$, which lands *almost* exactly on the gamma density, then repair the small discrepancy with a rejection step that accepts well over 95% of proposals. Shapes $k < 1$ (where the density blows up at zero) are handled by a classical boost: draw at shape $k + 1$ and multiply by $U^{1/k}$.

Origins and habitat. The name comes from Euler’s gamma function (1729), which normalises the density; as a *statistical* family it entered through Karl Pearson’s 1890s system of skew curves (his Type III), built to fit the asymmetric histograms — income, rainfall, biometry — that the normal manifestly could not. Erlang independently arrived at the integer-shape case in 1909 as the duration of k successive telephone holding times, and “Erlang distribution” survives in queueing to this day. The genesis mechanism is accumulation of positive quantities: total rainfall from many showers (the gamma is hydrology’s standard wet-day model), aggregate insurance losses, hospital lengths of stay, neuronal inter-spike intervals built from sub-threshold steps. Two structural roles keep it central to inference: it is the variance-component distribution behind §4.6, and in Bayesian statistics it is the conjugate prior for Poisson and exponential rates — a pairing systematised in the conjugate-family framework of [Raiffa and Schlaifer \(1961\)](#) — so that posterior inference about “how often” questions is, computationally, gamma sampling; the gamma-mixed Poisson also yields the negative binomial, the workhorse fix for the overdispersed counts of §3.4.

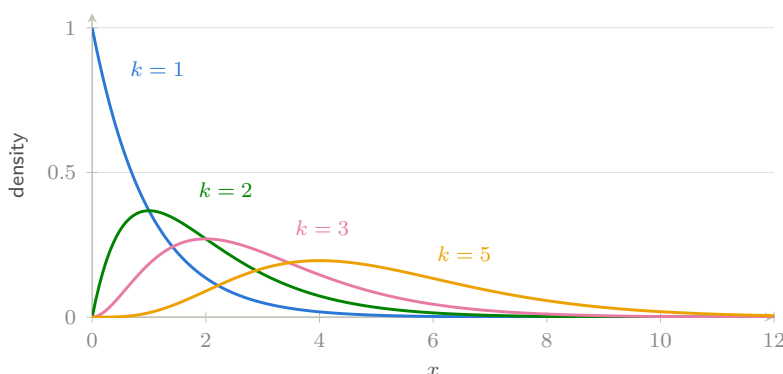


Figure 6: Gamma densities with scale $\theta = 1$. Shape $k = 1$ is the exponential; as k grows the density moves right, symmetrizes, and (by the CLT, since it is a sum of k waits) approaches a normal — the family tree of Figure 1 in a single picture. [[interactive version on the web](#)]

↔ implemented in `src/dist.c`: `ps_gamma()`, `ps_gamma_cdf()`

4.5 Beta

The $\text{Beta}(a, b)$ family lives on $(0, 1)$ with density $\propto x^{a-1}(1-x)^{b-1}$, mean $a/(a+b)$, and variance $ab/((a+b)^2(a+b+1))$; its shapes (Figure 7) run from uniform through unimodal humps to J- and U-curves. It is *the* distribution for unknown proportions: in Bayesian inference it is the conjugate prior for the Bernoulli/binomial p — start from $\text{Beta}(a, b)$, observe s successes and f failures, and the posterior is $\text{Beta}(a+s, b+f)$, which is why Thompson-sampling bandits and Bayesian A/B testing are, computationally, beta simulation. It also models proportions directly (beta regression) and is the order-statistics law: the k -th smallest of n uniforms is

$\text{Beta}(k, n-k+1)$. The sampler uses the classical gamma representation: $X = G_1/(G_1 + G_2)$ with independent $G_1 \sim \text{Gamma}(a, 1)$, $G_2 \sim \text{Gamma}(b, 1)$.

Origins and habitat. The beta is the oldest posterior in statistics. Thomas Bayes’s essay (Bayes, 1763), published posthumously by Richard Price, posed the inverse problem — given observed successes and failures, what is the chance itself? — through his billiard-table thought experiment, and the answer he computed is exactly the $\text{Beta}(s+1, f+1)$ update this section’s demo performs; Laplace, independently, integrated the same distribution to obtain his rule of succession (1774), the first published formula for learning a probability from data. Pearson later absorbed it as Type I of his curve system. It arises naturally three ways: as the law of an unknown proportion, as the k -th order statistic of n uniforms (so ranks, quantile estimates and records lead to it), and as the share $G_1/(G_1+G_2)$ of a gamma-distributed total — whence its multivariate sibling, the Dirichlet. Its application history has a distinctly modern arc: Thompson (1933) proposed allocating treatments by sampling from exactly this posterior, an idea that lay dormant for decades before returning as *Thompson sampling*, the algorithm inside contemporary bandit systems for ad serving, recommender exploration and adaptive clinical trials; the 1950s PERT method of project management adopted the beta for task durations, and empirical Bayes shrinkage of batting averages and hospital rates is Beta–binomial reasoning at scale.

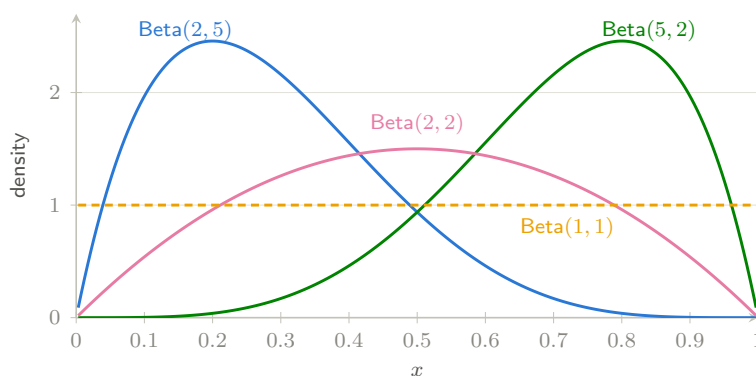


Figure 7: Beta densities: the driver’s worked example $\text{Beta}(2, 5)$ leans toward small proportions, its mirror $\text{Beta}(5, 2)$ toward large ones, $\text{Beta}(2, 2)$ is a symmetric hump, and $\text{Beta}(1, 1)$ is the uniform — the natural “know nothing” prior for a probability. [\[interactive version on the web\]](#)

↔ implemented in `src/dist.c`: `ps_beta()`, `ps_beta_cdf()`

4.6 Chi-squared

The χ_k^2 variable is a sum of k squared standard normals; it is also exactly $\text{Gamma}(k/2, 2)$, which is how the library both simulates it (one call to `ps_gamma`) and evaluates its CDF. Mean k , variance $2k$. Its starring role is as the *distribution of accumulated squared noise*: in a normal sample of size n , the rescaled sample variance $(n-1)S^2/\sigma^2$ is χ_{n-1}^2 — the fact that makes confidence intervals for variances possible and, in the next section, gives the t statistic its denominator. It reappears wherever squared deviations are summed: Pearson’s goodness-of-fit and independence tests, and the asymptotic null distribution of likelihood-ratio statistics (Wilks’ theorem) — which makes χ^2 tail areas the common currency of model comparison.

Origins and habitat. The distribution was first derived by the geodesist Friedrich Helmert (1876) while studying the sample variance of normal observations — surveying error analysis, not statistics, was the motivating problem. Its second birth was Pearson (1900): the goodness-of-fit

criterion $\sum(O - E)^2/E$, routinely listed among the most consequential papers in the history of science, because for the first time it let an experimenter ask, quantitatively, “does this model fit these data?” The famous 1922 dispute in which Fisher corrected Pearson’s degrees of freedom for contingency tables — estimate parameters, lose degrees — both fixed the practice and forged the modern concept of degrees of freedom. [Wilks \(1938\)](#) then made the χ^2 universal: twice the log likelihood ratio converges to it under the null, so every nested model comparison ultimately cashes out in χ^2 tail areas. In nature it appears wherever squared magnitudes of isotropic noise accumulate: the kinetic energy of an ideal-gas molecule is a scaled χ_3^2 (Maxwell), and the squared length of any k -dimensional Gaussian error vector — radar returns, portfolio residuals — is χ_k^2 by definition.

↪ implemented in `src/dist.c`: `ps_chi_squared()`, `ps_chi_squared_cdf()`

4.7 Student’s t

Student’s t_ν is defined by the construction at the heart of this library:

$$T = \frac{Z}{\sqrt{V/\nu}}, \quad Z \sim N(0, 1), \quad V \sim \chi_\nu^2 \text{ independent,}$$

and the sampler is that definition verbatim. It looks like a normal but with *heavier tails* (Figure 8): dividing by an estimated, rather than known, scale injects extra uncertainty, and the smaller ν is, the fatter the tails. As $\nu \rightarrow \infty$ it converges to the normal; at $\nu = 1$ it is the wild Cauchy with no mean at all. Historically it was discovered by [Student \(1908\)](#) — W. S. Gosset, publishing pseudonymously from the Guinness brewery — precisely because brewery samples were too small for large-sample normal theory. Beyond the t -tests of §5 (and the confidence intervals $\bar{x} \pm t_\nu^* s/\sqrt{n}$ dual to them), its heavy tails make it a workhorse *model* in its own right: robust regression replaces normal errors with t errors so outliers stop dragging the fit, and finance uses t innovations because asset returns produce extreme moves far more often than a Gaussian allows.

Origins and habitat. The backstory rewards telling. Gosset worked at Guinness on barley and hops experiments whose samples were unavoidably tiny, and large-sample normal theory visibly failed them; company policy forbade employees to publish (a rival brewery had been embarrassed by a leaked article), so his solution appeared under the pseudonym “Student”. The 1908 paper is also an early landmark of *simulation*: to check his curve, Gosset drew 750 samples of size four — by shuffling cards — from Macdonell’s anthropometric measurements of 3,000 convicts, a Monte Carlo experiment four decades before the name existed. The result was rigorous but little used until Fisher supplied a clean derivation, extended it to regression coefficients, and built his 1925 handbook around tabulated t ([Fisher, 1925](#)), turning small-sample exactness into everyday laboratory practice. Beyond testing, the t arises naturally as a *scale mixture* of normals — a Gaussian whose unknown variance is itself random — which is precisely why it suits populations that are normal-with-heterogeneous-precision: measurement series with occasional sloppy runs, asset returns alternating between calm and turbulent regimes, and any Bayesian analysis where integrating out an unknown variance turns a normal likelihood into a t predictive.

↪ implemented in `src/dist.c`: `ps_student_t()`, `ps_student_t_pdf()`, `ps_student_t_cdf()`

4.8 Fisher’s F

The F_{d_1, d_2} variable is a ratio of independent normalized chi-squareds, $F = (U/d_1)/(V/d_2)$ — the sampler, once again, is the definition. Since sample variances are scaled chi-squareds (§4.6), F is the natural null distribution for *comparing variances*: its tail areas power the classical

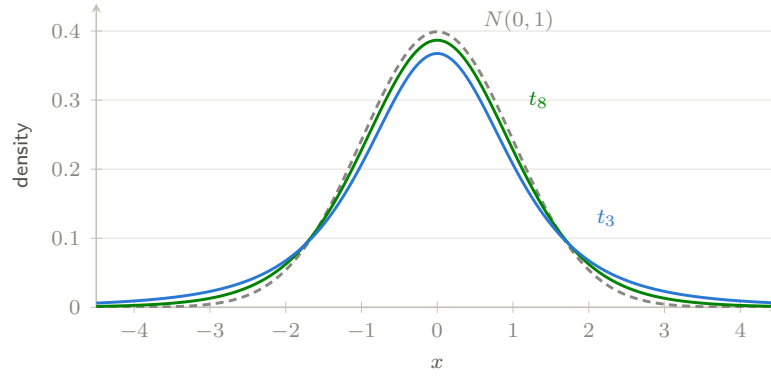


Figure 8: Student’s t against the standard normal. The lower the degrees of freedom, the more probability escapes into the tails — so small samples need larger observed effects before a result counts as surprising. This gap between t_ν and $N(0,1)$ is the small-sample correction of §5. [\[interactive version on the web\]](#)

ANOVA F -test (is the variance *between* group means larger than the variance *within* groups?), the overall significance test of a regression, and nested-model comparisons. Two connections knit the family shut: $t_\nu^2 = F_{1,\nu}$ (squaring a t -test gives an F -test), and ANOVA with two groups is exactly the pooled t -test.

Origins and habitat. The F is the youngest of the family and the most deliberately engineered. Fisher developed the analysis of variance in the 1920s at Rothamsted Experimental Station, where decades of agricultural field trials — plots of wheat under different manures, varieties and years — demanded a way to split observed variation into components and judge which differences exceeded chance; his 1925 handbook (Fisher, 1925) and the later *Design of Experiments* (1935) wrapped the variance ratio in the trio that still defines good experimentation: randomization, replication, and blocking. Fisher himself worked with $z = \frac{1}{2} \ln F$; George Snedecor’s 1934 tables at Iowa State popularised the untransformed ratio and named it F in Fisher’s honour. It arises wherever two independent estimates of noise compete: signal variance against residual variance in ANOVA and regression, a smaller model’s leftover error against a larger model’s in nested comparisons, and channel power against background in variance-ratio detectors. Its $t^2 = F_{1,\nu}$ identity is the formal statement that the t -test is one-factor ANOVA in miniature — the whole classical testing toolkit is, in the end, ratios of chi-squareds.

↔ implemented in `src/dist.c`: `ps_f()`, `ps_f_cdf()`

4.9 Rayleigh, and the birthday bound

The last two members of our collection are less common in introductory courses but indispensable in one modern habitat: modeling *cryptographic hash functions*. An ideal n -bit hash behaves as a random oracle — each input is a ball thrown uniformly into $N = 2^n$ bins — so its security statistics are the statistics of balls in bins, and those statistics are governed by exactly the distributions of this and the next section.

The Rayleigh(σ) distribution has density, mean and variance

$$f(x) = \frac{x}{\sigma^2} e^{-x^2/2\sigma^2} \quad (x \geq 0), \quad \mathbb{E}X = \sigma\sqrt{\pi/2}, \quad \text{Var } X = \frac{4 - \pi}{2} \sigma^2,$$

and two equivalent geneses: it is the length $\sqrt{Z_1^2 + Z_2^2}$ of a two-dimensional Gaussian vector (a scaled χ with 2 df), and it is $\sigma\sqrt{2E}$ for a standard exponential E — the one-line inversion sampler. Its cryptographic starring role is the *birthday problem*: throw balls into N bins until

two share a bin, and the waiting time T satisfies $T/\sqrt{N} \rightarrow \text{Rayleigh}(1)$ as $N \rightarrow \infty$. Hence collisions in an ideal n -bit hash appear after about $\sqrt{\pi N/2} \approx 1.25 \cdot 2^{n/2}$ evaluations — the *birthday bound* that halves every hash’s collision resistance relative to its output size, dictates that 128-bit hashes are collision-broken at 2^{64} work, and sets the $\sqrt{N}$ running time of Pollard (1975)-style rho methods, whose cycle-finding walks are governed by the same law. The analysis of these collision phenomena is classical (Knuth, 1997, §6.4).

Origins and habitat. Lord Rayleigh derived the distribution in 1880 while asking what amplitude results from superposing many acoustic vibrations of random phase (Rayleigh, 1880) — a two-dimensional random walk in disguise. The same geometry makes it ubiquitous wherever a resultant of many random phasors is measured: multipath radio propagation (*Rayleigh fading*, the baseline channel model of mobile communications), radar clutter amplitudes, sea-wave heights, and wind speeds; and wherever squared Gaussian components pair up — speckle in ultrasound and synthetic-aperture imagery. In the hash-modeling annex it is the distribution to simulate when estimating collision-search cost under resource constraints, as the driver’s Rayleigh row does in miniature.

↪ implemented in `src/dist.c`: `ps_rayleigh()`, `ps_rayleigh_pdf()`, `ps_rayleigh_cdf()`

4.10 Gumbel, and the fullest bucket

Where the Rayleigh governs the *first* coincidence, the Gumbel(μ, β) distribution governs the *worst* one. With CDF and moments

$$F(x) = \exp(-e^{-(x-\mu)/\beta}), \quad \mathbb{E}X = \mu + \gamma\beta \ (\gamma \approx 0.5772), \quad \text{Var } X = \frac{\pi^2}{6}\beta^2,$$

it is the extreme-value limit: by the Fisher–Tippett theorem (Fisher and Tippett, 1928), suitably centred maxima of independent light-tailed draws (normal, exponential, gamma, ...) converge to it — the CLT’s counterpart for maxima rather than sums. The canonical concrete case is exponential draws: $\max(E_1, \dots, E_n) - \ln n \rightarrow \text{Gumbel}(0, 1)$, which the web edition’s demo simulates live. For hash modeling it answers “how bad is the worst case?”: the most-loaded bucket of a hash table, the longest probe sequence, and the longest run of identical bits in an n -bit output stream — approximately $\log_2 n$ plus Gumbel-type fluctuation (up to lattice effects), the null distribution behind the longest-run-of-ones test in NIST’s randomness battery for cryptographic generators (Rukhin et al., 2010). Sampling is double-log inversion, $X = \mu - \beta \ln(-\ln U)$.

Origins and habitat. Extreme-value theory grew from a practical question — how large a flood must a dam survive? — and Emil Gumbel’s *Statistics of Extremes* (Gumbel, 1958) turned the Fisher–Tippett limit into an engineering discipline; annual-maximum river discharges, extreme winds, and design-basis earthquakes are still fitted with his distribution (Gumbel himself was also a courageous public statistician, driven from Germany for documenting political murders). Beyond civil engineering it models record values in athletics and climate, peak electricity demand, and maximal sequence-alignment scores in bioinformatics (BLAST E -values are Gumbel tail areas); machine learning rediscovered it as the *Gumbel-max trick*, which turns softmax sampling into an argmax of Gumbel-perturbed scores. It is the right habitat whenever the quantity reported is a maximum over many comparable chances — which is what worst-case analysis of hashing is.

↪ implemented in `src/dist.c`: `ps_gumbel()`, `ps_gumbel_pdf()`, `ps_gumbel_cdf()`

5 From distributions to inference: the t-test

Everything so far generated data; this section reasons *backwards* from data. The *t*-test is the classical family’s flagship application, and its logic recurs across all of statistics: reduce the data to a statistic whose distribution under a null hypothesis is known exactly (here, thanks to §4.3–§4.7), then ask how extreme the observed value is under that distribution.

5.1 Two special functions carry all the tables

Before software, the answer to “how extreme?” lived in printed tables at the back of textbooks. All of those tables compress into two special functions. The *regularized incomplete beta function*

$$I_x(a, b) = \frac{1}{B(a, b)} \int_0^x u^{a-1} (1-u)^{b-1} du, \quad (5.1)$$

is the beta CDF, and the *t* and *F* CDFs reduce to it algebraically; for the *t*,

$$\Pr[T_\nu \leq t] = 1 - \frac{1}{2} I_{\frac{\nu}{\nu+t^2}}\left(\frac{\nu}{2}, \frac{1}{2}\right) \quad (t \geq 0), \quad (5.2)$$

with the $t < 0$ case supplied by symmetry, and $\Pr[F_{d_1, d_2} \leq x] = I_{d_1 x / (d_1 x + d_2)}(d_1/2, d_2/2)$. Its sibling, the *regularized lower incomplete gamma function* $P(a, x) = \frac{1}{\Gamma(a)} \int_0^x u^{a-1} e^{-u} du$, is the gamma CDF and hence the chi-squared and Poisson tail machinery. **probsim** evaluates both with the classical series/continued-fraction pairs (Press et al., 1992, §6.2, §6.4), accurate to about 10^{-14} — each roughly forty lines of C that replace every table in the book.

↪ implemented in `src/special.c`: `ps_incbeta()`, `ps_incgamma_lower()`

5.2 The one-sample t-test

Model n observations as $X_i \sim N(\mu, \sigma^2)$ with both parameters unknown, and test $H_0: \mu = \mu_0$. Three facts from §4 assemble the test. Under H_0 : (i) $\bar{X} \sim N(\mu_0, \sigma^2/n)$; (ii) $(n-1)S^2/\sigma^2 \sim \chi_{n-1}^2$, where S^2 is the sample variance with the $n-1$ divisor — exactly the divisor that makes this distributional statement true, which is why **ps_summarize** uses it; (iii) $\bar{X}$ and S^2 are independent. Standardizing $\bar{X}$ and substituting S for the unknown σ produces precisely the $Z/\sqrt{V/\nu}$ construction of §4.7:

$$T = \frac{\bar{X} - \mu_0}{S/\sqrt{n}} \sim t_{n-1} \quad \text{under } H_0. \quad (5.3)$$

The unknown σ cancels — that is the miracle Gosset found. The two-sided *p*-value is then a pure CDF evaluation, $p = 2 F_{t_{n-1}}(-|t_{\text{obs}}|)$, via equation (5.2):

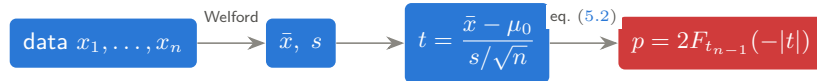


Figure 9 shows the geometry on the library’s own test fixture (§6.3): ten measurements with mean 5.47, tested against $\mu_0 = 5$, give $t_{\text{obs}} = 3.12$ on 9 degrees of freedom and $p = 0.012$ — the observed statistic sits well beyond the 5% critical value, so either H_0 is false or a 1-in-80 coincidence occurred. The summaries themselves are computed by Welford’s one-pass recurrence (Welford, 1962), which avoids the catastrophic cancellation of the naive sum-of-squares formula.

↪ implemented in `src/stats.c`: `ps_summarize()`, `ps_t_test_one_sample()`

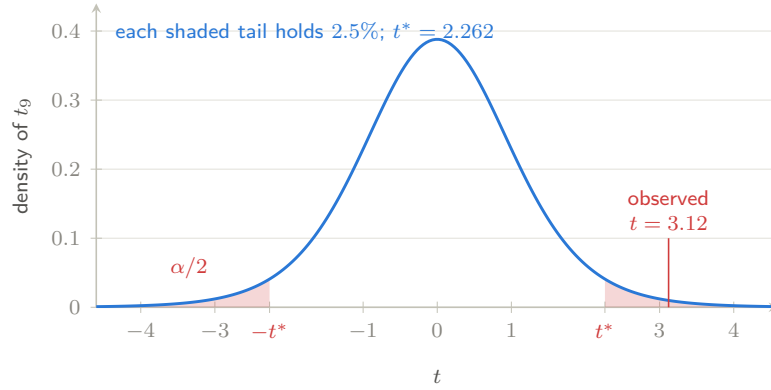


Figure 9: The one-sample t -test on the repository’s own test fixture ($n = 10$, $\text{df} = 9$). Under H_0 the statistic follows the blue t_9 density; the shaded tails beyond $\pm t^*$ are the 5% rejection region, and the observed $t = 3.12$ lands past them, giving the two-sided p -value 0.012 — the total tail area beyond ± 3.12 . [\[interactive version on the web\]](#)

5.3 Welch’s two-sample t -test

The commonest real question is comparative: do two groups — treatment and control, variant A and variant B — share a mean? With samples $x_1, \dots, x_{n_x}$ and $y_1, \dots, y_{n_y}$, [Welch \(1947\)](#) tests $H_0: \mu_X = \mu_Y$ with

$$t = \frac{\bar{x} - \bar{y}}{\sqrt{s_x^2/n_x + s_y^2/n_y}}, \quad \nu \approx \frac{(s_x^2/n_x + s_y^2/n_y)^2}{\frac{(s_x^2/n_x)^2}{n_x - 1} + \frac{(s_y^2/n_y)^2}{n_y - 1}}, \quad (5.4)$$

the degrees of freedom being the Welch–Satterthwaite approximation ([Satterthwaite, 1946](#)) — generally fractional, which is why `ps_t_test` carries `df` as a double. Unlike the older pooled test, Welch’s does not assume the two variances are equal, and it loses almost nothing when they happen to be; that robustness has made it the default two-sample test in modern practice (and in R’s `t.test`). It is the standard analysis for randomized experiments and A/B tests; the same statistic inverted gives the confidence interval for the difference in means that any experiment report should carry.

↪ implemented in `src/stats.c: ps_t_test_welch()`

6 The driver and the tests

The repository ships two executables that turn the theory above into running evidence: a driver that *shows* the library working, and a test suite that *proves* it.

6.1 Simulation meets theory

The driver (`make run`) simulates 200,000 draws from each distribution at the worked parameters of §3–§4 and prints sample mean and variance beside the theoretical $\mathbb{E}X$ and $\text{Var } X$ — the law of large numbers as a table (excerpt from seed 20260718):

distribution	mean	$\mathbb{E}X$	variance	$\text{Var } X$
Binomial(20, 0.25)	5.008	5	3.733	3.75
Poisson(4)	3.990	4	3.990	4
Exponential(0.5)	1.995	2	3.985	4
Normal(10, 2)	10.001	10	3.985	4
Gamma(3, 2)	6.009	6	12.002	12
Beta(2, 5)	0.285	0.286	0.0254	0.0255
Student's t_8	-0.002	0	1.332	4/3
$F_{5,12}$	1.201	1.2	1.077	1.08

6.2 The t-tests, live

The driver then runs both tests of §5 on data it has just simulated — a situation where, uniquely, we *know* the truth. A one-sample test of $N(10, 2)$ data against its true mean (H_0 true) yielded $p = 0.44$: no false alarm. A Welch test of $N(10, 2)$ against $N(11.5, 3)$ (H_0 false) yielded $p = 0.014$: difference detected. Re-running with other seeds shows the expected behaviour: under a true null the p -values scatter uniformly (occasionally below 0.05 — type I error is a feature, not a bug), while under the false null they concentrate near zero at this effect size and n .

6.3 The test suite

make test runs about a hundred thousand checks in three layers, in increasing order of subtlety. *Fixtures*: CDF, pmf and t -test results are compared to reference values computed independently with SciPy (each fixture's provenance is recorded beside it in the source). *Identities*: mathematical facts that must hold exactly, such as the symmetry $I_x(a, b) + I_{1-x}(b, a) = 1$ of equation (5.1), or $P(\frac{1}{2}, x) = \text{erf } \sqrt{x}$ linking our incomplete gamma to the C library's **erf**. *Statistical*: with a fixed seed, every sampler's 200,000-draw mean must land within five standard errors of theory; and 2,000 simulated one-sample t -tests under a true null must reject at close to the nominal 5% rate — an end-to-end check that samplers, special functions and test logic agree, and an empirical verification of the uniform- p -value fact of §4.1.

↪ implemented in `app/simulate.c` and `tests/test_probsim.c`

7 A modeler's cheat sheet

The table below compresses the tour into one glance: what each distribution models, and the inference task where it most often appears.

distribution	models	flagship inference role
Bernoulli	single yes/no event	proportion estimation, A/B tests
Binomial	successes in n trials	exact tests of proportions
Geometric	failures until success	retry/attrition modeling
Poisson	rare-event counts	count regression, goodness of fit
Uniform	total ignorance on a range	p -values under H_0 ; simulation
Exponential	memoryless waiting time	survival & reliability baselines
Normal	sums of small effects	CLT; regression noise
Gamma	accumulated waits, scales	conjugate prior for rates
Beta	unknown proportions	conjugate prior for p ; bandits
χ^2	summed squared noise	variance CIs, GoF, likelihood ratios
Student t	normalized small samples	t -tests, robust models
F	variance ratios	ANOVA, regression F -tests
Neg. binomial	failures before r -th success	overdispersed counts; multi-collision cost
Rayleigh	2-D Gaussian lengths	birthday bound; fading channels
Gumbel	maxima of many draws	worst-case loads; extremes; NIST runs test

For going further, three references span the territory: [Devroye \(1986\)](#) for everything about generating non-uniform random variables, [Knuth \(1997\)](#) for the theory of the uniform generators underneath, and [Press et al. \(1992\)](#) for the numerical special-function craft of §5.1.

References

- Thomas Bayes. An essay towards solving a problem in the doctrine of chances. *Philosophical Transactions of the Royal Society of London*, 53:370–418, 1763. doi: 10.1098/rstl.1763.0053. Communicated by Richard Price. Local copy: [refs/bayes-1763.pdf](#) (open access).
- Jacob Bernoulli. *Ars Conjectandi*. Thurneysen Brothers, Basel, 1713. Published posthumously; contains the first law of large numbers.
- David Blackman and Sebastiano Vigna. Scrambled linear pseudorandom number generators. *ACM Transactions on Mathematical Software*, 47(4):1–32, 2021. doi: 10.1145/3460772. Preprint arXiv:1805.01407; local copy: [refs/blackman-vigna-xoshiro.pdf](#).
- George E. P. Box and Mervin E. Muller. A note on the generation of random normal deviates. *The Annals of Mathematical Statistics*, 29(2):610–611, 1958. doi: 10.1214/aoms/1177706645. Local copy: [refs/box-muller-1958.pdf](#) (open access).
- Abraham de Moivre. *The Doctrine of Chances*. Woodfall, London, 2nd edition, 1738. Incorporates the 1733 normal approximation to the binomial.
- Luc Devroye. *Non-Uniform Random Variate Generation*. Springer, New York, 1986. doi: 10.1007/978-1-4613-8643-8. Freely available chapter-by-chapter from the author at luc.devroye.org/rnbookindex.html.
- Chris Doty-Humphrey. PractRand: Practically random — C++ library of statistical tests for RNGs. <https://pracrand.sourceforge.net>, 2014.
- Ronald A. Fisher. *Statistical Methods for Research Workers*. Oliver and Boyd, Edinburgh, 1925.
- Ronald A. Fisher and Leonard H. C. Tippett. Limiting forms of the frequency distribution of the largest or smallest member of a sample. *Mathematical Proceedings of the Cambridge Philosophical Society*, 24(2):180–190, 1928. doi: 10.1017/S0305004100015681.

- Carl Friedrich Gauss. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Perthes and Besser, Hamburg, 1809.
- Major Greenwood and G. Udny Yule. An inquiry into the nature of frequency distributions representative of multiple happenings with particular reference to the occurrence of multiple attacks of disease or of repeated accidents. *Journal of the Royal Statistical Society*, 83(2): 255–279, 1920. doi: 10.2307/2341080.
- Emil J. Gumbel. *Statistics of Extremes*. Columbia University Press, New York, 1958.
- Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd edition, 1997.
- Pierre-Simon Laplace. *Théorie analytique des probabilités*. Courcier, Paris, 1812.
- Pierre L’Ecuyer and Richard Simard. TestU01: A C library for empirical testing of random number generators. *ACM Transactions on Mathematical Software*, 33(4):22:1–22:40, 2007. doi: 10.1145/1268776.1268777.
- George Marsaglia. Xorshift RNGs. *Journal of Statistical Software*, 8(14):1–6, 2003. doi: 10.18637/jss.v008.i14. Local copy: [refs/marsaglia-2003-xorshift.pdf](#) (open access).
- George Marsaglia and Wai Wan Tsang. A simple method for generating gamma variables. *ACM Transactions on Mathematical Software*, 26(3):363–372, 2000a. doi: 10.1145/358407.358414.
- George Marsaglia and Wai Wan Tsang. The ziggurat method for generating random variables. *Journal of Statistical Software*, 5(8):1–7, 2000b. doi: 10.18637/jss.v005.i08. Local copy: [refs/marsaglia-tsang-ziggurat-2000.pdf](#) (open access).
- Melissa E. O’Neill. PCG: A family of simple fast space-efficient statistically good algorithms for random number generation. Technical Report HMC-CS-2014-0905, Harvey Mudd College, 2014. Local copy: [refs/oneill-pcg-2014.pdf](#).
- Karl Pearson. On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *Philosophical Magazine, Series 5*, 50(302):157–175, 1900. doi: 10.1080/14786440009463897.
- John M. Pollard. A Monte Carlo method for factorization. *BIT Numerical Mathematics*, 15(3): 331–334, 1975. doi: 10.1007/BF01933667.
- William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes in C: The Art of Scientific Computing*. Cambridge University Press, 2nd edition, 1992.
- Howard Raiffa and Robert Schlaifer. *Applied Statistical Decision Theory*. Harvard University, Graduate School of Business Administration, Boston, 1961.
- Lord (John William Strutt) Rayleigh. On the resultant of a large number of vibrations of the same pitch and of arbitrary phase. *Philosophical Magazine, Series 5*, 10(60):73–78, 1880. doi: 10.1080/14786448008626893.
- Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, et al. A statistical test suite for random and pseudorandom number generators for cryptographic applications. Technical Report Special Publication 800-22, Revision 1a, National Institute of Standards and Technology, 2010. Local copy: [refs/nist-sp800-22r1a.pdf](#) (open access).

- Franklin E. Satterthwaite. An approximate distribution of estimates of variance components. *Biometrics Bulletin*, 2(6):110–114, 1946. doi: 10.2307/3002019.
- Student. The probable error of a mean. *Biometrika*, 6(1):1–25, 1908. doi: 10.1093/biomet/6.1.1. Local copy: `refs/student-1908.pdf` (public-domain scan).
- William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933. doi: 10.1093/biomet/25.3-4.285.
- Sebastiano Vigna. An experimental exploration of Marsaglia’s xorshift generators, scrambled. *ACM Transactions on Mathematical Software*, 42(4):30:1–30:23, 2016. doi: 10.1145/2845077. Preprint arXiv:1402.6246.
- Ladislav von Bortkiewicz. *Das Gesetz der kleinen Zahlen*. Teubner, Leipzig, 1898.
- Bernard L. Welch. The generalization of “Student’s” problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, 1947. doi: 10.1093/biomet/34.1-2.28.
- B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962. doi: 10.1080/00401706.1962.10490022.
- Samuel S. Wilks. The large-sample distribution of the likelihood ratio for testing composite hypotheses. *The Annals of Mathematical Statistics*, 9(1):60–62, 1938. doi: 10.1214/aoms/1177732360. Local copy: `refs/wilks-1938.pdf` (open access).